Section 14 : Operator Overloading

Information

- Title: `Operator Overloading`
- Authors: `Etienne KOA`

Aperçu

Surcharge de l'opérateur - Qu'est-ce que la surcharge des opérateurs ? - Surcharge de l'opérateur d'affectation `(=)` - Copier la sémantique - Déplacer la sémantique

- Surcharge des opérateurs en tant que fonctions membres
- Surcharge des opérateurs en tant que fonctions globales
- Surcharge des opérateurs d'insertion de flux `(<<)` et d'extraction `(>>)`

Surcharge des opérateurs

Qu'est-ce que la surcharge des opérateurs ?

- Utilisation de traditionnels tels que `+,=,*,etc.` avec des types définis par l'utilisateur.
- Permet aux types définis par l'utilisateur de se comporter de manière similaire aux types intégrés
- Peut rendre le code plus lisible et inscriptible
- Pas fait automatiquement (sauf pour l'opérateur d'affectation). Ils doivent être explicitement définis.

Qu'est-ce que la surcharge des opérateurs ?

- Supposons que nous ayons une classe Number qui modélise n'importe quel nombre
- Utilisation des fonctions :

```
Number result = multiply (add(a, b), divide(c, d));
```

- Utilisation des méthodes membres :

```
Number result = (a.add(b).multiply(c.divide(d)));
```

- Utilisation d'opérateurs surchargés

```
Number result = (a+b)*(c/d);
```

Quels opérateurs peuvent être surchargés ? + La majorité des opérateurs `C++` peuvent être surchargés + Les opérateurs suivants ne peuvent pas être surchargés

Quelques règles de base + La priorité et l'associativité ne peuvent pas être modifiées + `arity` ne peut pas être modifié (c'est-à-dire ne peut pas rendre

operator
::
:?
.*
.
sizeof

Figure 1: Operator_Overloading

l'opérateur de division unaire) + Impossible de surcharger les opérateurs pour les types privilégiés (par exemple `int, double, etc.`) + Impossible de créer de nouveaux opérateurs + `[], (), ->`, et l'opérateur d'affectation `(=) doivent` être déclarés comme méthodes membres. + D'autres opérateurs peuvent être déclarés comme méthodes membres ou fonctions globales.

Quelques exemples

- **int**
  ```
  a = b + c
  a < b
  std::cout << a
  ```
- **double**
  ```
  a = b + c
  a < b
  std::cout << a
  ```
- **long**
  ```
  a = b + c
  a < b
  std::cout << a
  ```
- **std::string**
  ```
  s1 = s2 + s3
  S1 < s2
  std::cout << s1
  ```
- **Mystring**
  ```
  s1 = s2 + s3
  s1 < s2
  s1 == s2
  std::cout << s1
  ```
- **Player**
  ```
  p1 < p2
  p1 == p2
  std::cout << p1
  ```

Figure 2: Operator_Overloading

Déclaration de classe Mystring

```
class Mystring {

private:
    char *str; // C-style string

public:
    Mystring();
    Mystring(const char *s);
    Mystring(const Mystring &source);
    ~Mystring();
    void display() const;
    int get_length() const;
    const char *get_str() const;
};
```

Figure 3: Operator_Overloading

Surcharge des opérateurs

Copier l'opérateur d'affectation (=)

- `C++` fournit un opérateur d'affectation par défaut utilisé pour attribuer un

```
Mystring s1 {"Frank"};
Mystring s2 = s1;        // NOT assignment
                    // same as Mystring s2{s1};

s2 = s1;            // assignment
```

objet à un autre

- La valeur par défaut est l'affectation par membre (copie superficielle)
  - Si nous avons un membre de données de pointeur brut, nous devons copier en profondeur

Surcharge de l'opérateur d'affectation de copie (copie profonde)

```
Type &Type::operator=(const Type &rhs);
```

```
Mystring &Mystring::operator=(const Mystring &rhs);

s2 = s1;               // We write this

s2.operator=(s1);      // operator= method is called
```

```
Mystring &Mystring::operator=(const Mystring &rhs) {
   if (this == &rhs)
      return *this;

   delete [] str;
   str = new char[std::strlen(rhs.str) + 1];
   std::strcpy(str, rhs.str);

   return *this;
}
```

- Surcharge de l'opérateur d'affectation de copie - étapes pour la copie complète

```
if (this == &rhs)      // p1 = p1;
   return *this;       // return current object
```

  - Vérifier l'auto-affectation
  - Désallouer le stockage pour `this -> str` puisque nous l'écrasons

```
delete [] str;
```

  - Allouer du stockage pour la copie complète `str = new char[std::strlen(rhs.str) + 1];`
  - Effectuer la copie `std::strcpy(str, rhs.str);`

– Renvoie le courant par référence pour permettre l'affectation de la chaîne

```
return *this;

// s1 = s2 = s3;
```

Surcharge de l'affectation (déplacement de l'opérateur)

Déplacer l'opérateur d'affectation (=)

- Vous pouvez choisir de surcharger l'opérateur d'affectation de déplacement
  - `C++` utilisera l'opérateur d'affectation de copie si nécessaire

```
Mystring s1;

s1 = Mystring {"Frank"}; // move assignment
```

- Si nous avons un pointeur brut, nous devrions surcharger l'opérateur d'affectation de déplacement pour plus d'efficacité
- Surcharge de l'opérateur d'affectation Move

```
Type &Type::operator=(Type &&rhs);


Mystring &Mystring::operator=(Mystring &&rhs);

s1 = Mystring{"Joe"}; // move operator= called

s1 = "Frank";         // move operator= called
```

```
Mystring &Mystring::operator=(Mystring &&rhs) {

    if (this == &rhs)     // self assignment
       return *this;      // return current object

    delete [] str;        // deallocate current storage
    str = rhs.str;        // steal the pointer

    rhs.str = nullptr;    // null out the rhs object

    return *this;         // return current object
}
```

- Surcharge de l'opérateur d'affectation Move - étapes
  - Vérifier l'auto-affectation

```
if (this == &rhs)
   return *this;         // return current object
```

- Désallouer le stockage `this -> str` puisque nous l'écrasons

  ```
  delete [] str;
  ```

- Surcharge de l'opérateur d'affectation Move - étapes pour la copie complète
  - Volez le pointeur de l'objet `rhs` et assignez-le à `this->str`

    ```
    str = rhs.str;
    ```

  - Annuler le pointeur `rhs` `rhs.str = nullptr;`
  - Renvoie l'objet actuel par référence pour permettre l'affectation de la chaîne `return *this;`

Surcharge des opérateurs en tant que fonctions membres

Surcharge de l'opérateur

- Opérateurs unaires en tant que méthodes membres `(++, --, -, !)`

```
ReturnType Type::operatorOp();

Number Number::operator-() const;
Number Number::operator++();        // pre-increment
Number Number::operator++(int);     // post-increment
bool Number::operator!() const;

Number n1 {100};
Number n2 = -n1;                    // n1.operator-()
n2 = ++n1;                          // n1.operator++()
n2 = n1++;                          // n1.operator++(int)
```

Figure 4: Operator_Overloading

- Mystring `operator` - mettre en minuscule

```
Mystring larry1 {"LARRY"};
Mystring larry2;

larry1.display();          // LARRY

larry2 = -larry1;          // larry1.operator-()

larry1.display();          // LARRY
larry2.display();          // larry
```

```
Mystring Mystring::operator-() const {
   char *buff = new char[std::strlen(str) + 1];
   std::strcpy(buff, str);
   for (size_t i=0; i<std::strlen(buff); i++)
      buff[i] = std::tolower(buff[i]);
   Mystring temp {buff};
   delete [] buff;
   return temp;
}
```

- Opérateurs binaires comme méthode membre `(+, -, ==, !=, <,>, etc.)`

```
ReturnType Type::operatorOp(const Type &rhs);

Number Number::operator+(const Number &rhs) const;
Number Number::operator-(const Number &rhs) const;
bool Number::operator==(const Number &rhs) const;
bool Number::operator<(const Number &rhs) const;

Number n1 {100}, n2 {200};
Number n3 = n1 + n2;                // n1.operator+(n2)
n3 = n1 - n2;                       // n1.operator-(n2)
if (n1 == n2) . . .                 // n1.operator==(n2)
```

Figure 5: Operator_Overloading

- Mystring `opérateur ==`
- Mystring `opérateur + (concaténation)`

```
bool Mystring::operator==(const Mystring &rhs) const {
   if (std::strcmp(str, rhs.str) == 0)
      return true;
   else
      return false;
}

// if (s1 == s2)        // s1 and s2 are Mystring objects
```

Figure 6: Operator_Overloading

```
Mystring larry {"Larry"};
Mystring moe{"Moe"};
Mystring stooges {" is one of the three stooges"};

Mystring result = larry + stooges;
                // larry.operator+(stooges);

result = moe +  " is also a stooge";
                // moe.operator+("is also a stooge");

result = "Moe" + stooges; // "Moe".operator+(stooges) ERROR

Mystring Mystring::operator+(const Mystring &rhs) const {
   size_t buff_size = std::strlen(str) +
                      std::strlen(rhs.str) + 1;
   char *buff = new char[buff_size];
   std::strcpy(buff, str);
   std::strcat(buff, rhs.str);
   Mystring temp {buff};
   delete [] buff;
   return temp;
}
```

Surcharge des opérateurs en tant que fonctions globales

- Opérateurs unaires en tant que fonctions globales (`++`, `--`, `-`, `!`)
- Mystring `operator` - mettre en minuscule

Mystring `opérateur -` + Souvent déclaré comme fonctions `friend` dans la déclaration de classe

- Opérateurs binaires comme fonctions globales (`+`, `-`, `==`, `!=`, `<`,`>`, `etc.`)

```
ReturnType operatorOp(Type &obj);

Number operator-(const Number &obj);
Number operator++(Number &obj);          // pre-increment
Number operator++(Number &obj, int);     // post-increment
bool   operator!(const Number &obj);

Number n1 {100};
Number n2 = -n1;                    // operator-(n1)
n2 = ++n1;                          // operator++(n1)
n2 = n1++;                          // operator++(n1,int)
```

Figure 7: Operator_Overloading

```
Mystring larry1 {"LARRY"};
Mystring larry2;

larry1.display();                   // LARRY

larry2 = -larry1;                   // operator-(larry1)

larry1.display();                   // LARRY
larry2.display();                   // larry
```

Figure 8: Operator_Overloading

```
Mystring operator-(const Mystring &obj) {
   char *buff = new char[std::strlen(obj.str) + 1];
   std::strcpy(buff, obj.str);
   for (size_t i=0; i<std::strlen(buff); i++)
      buff[i] = std::tolower(buff[i]);
   Mystring temp {buff};
   delete [] buff;
   return temp;
}
```

Figure 9: Operator_Overloading

```
ReturnType operatorOp(const Type &lhs, const Type &rhs);

Number operator+(const Number &lhs, const Number &rhs);
Number operator-(const Number &lhs, const Number &rhs);
bool   operator==(const Number &lhs, const Number &rhs);
bool   operator<(const Number &lhs, const Number &rhs);

Number n1 {100}, n2 {200};
Number n3 = n1 + n2;                // operator+(n1,n2)
n3 = n1 - n2;                       // operator-(n1,n2)
if (n1 == n2) . . .                 // operator==(n1,n2)
```

Figure 10: Operator_Overloading

- Mystring`opérateur ==`

```
bool operator==(const Mystring &lhs, const Mystring &rhs){
   if (std::strcmp(lhs.str, rhs.str) == 0)
      return true;
   else
      return false;
}
```

Figure 11: Operator_Overloading

- S'il est déclaré comme ami de `Mystring`, il peut accéder à l'attribut privé `str`.
- Sinon, nous devons utiliser des méthodes getter.
- Mystring `opérateur + (concaténation)`

```
Mystring larry {"Larry"};
Mystring moe{"Moe"};
Mystring stooges {" is one of the three stooges"};

Mystring result = larry + stooges;
               // operator+(larry, stooges);

result = moe +  " is also a stooge";
               // operator+(moe, "is also a stooge");

result = "Moe" + stooges; // OK with non-member functions
```

```
Mystring operator+(const Mystring &lhs, const Mystring &rhs) {
   size_t buff_size = std::strlen(lhs.str) +
                      std::strlen(rhs.str) + 1;
   char *buff = new char[buff_size];
   std::strcpy(buff, lhs.str);
   std::strcat(buff, rhs.str);
   Mystring temp {buff};
   delete [] buff;
   return temp;
}
```

Surcharge des opérateurs d'insertion et d'extraction de flux

Surcharge de l'opérateur

- opérateurs d'insertion et d'extraction de flux (`<<`, `>>`)

```
Mystring larry {"Larry"};

cout << larry << endl;   // Larry


Player hero {"Hero", 100, 33};

cout << hero << endl;   // {name: Hero, health: 100, xp:33}

  Mystring larry;

  cin >> larry;


  Player hero;

  cin >> hero;
```

- Cela n'a pas de sens à implémenter en tant que méthodes membres
  - L'opérande Lfet doit être une classe définie par l'utilisateur
  - Pas la façon dont nous utilisons habituellement ces opérateurs
- Opérateur d'insertion de flux (`<<`)
- Renvoie une référence au `ostream` afin que nous puissions continuer à insérer
- Ne renvoie pas `ostream` par valeur !
- Opérateur d'extraction de flux (`>>`)

```
Mystring larry;
larry << cout;         // huh?

Player hero;
hero >> cin;           // huh?
```

Figure 12: Operator_Overloading

```
std::ostream &operator<<(std::ostream &os, const Mystring &obj) {
    os << obj.str;       // if friend function
    // os << obj.get_str();  // if not friend function
    return os;
 }
```

Figure 13: Operator_Overloading

```
std::istream &operator>>(std::istream &is, Mystring &obj) {
    char *buff = new char[1000];
    is >> buff;
    obj = Mystring{buff};// If you have copy or move assignment
    delete [] buff;
    return is;
 }
```

Figure 14: Operator_Overloading

- Renvoie une référence au `istream` afin que nous puissions continuer à insérer
- Mettre à jour l'objet transmis